AI PERSONAL STUDY ASSISTANT

PYTHON STUDY MATERIAL

A beginner-friendly 9-page guide with explanations, code examples, revision notes and practice activities.

Name : Alan S Riju

Department: Computer Science(AIML)

LEARNING ROADMAP

PAGE	UNIT	CORE CONTENT
2	Python Foundations	Overview, uses, syntax and first program
3	Variables & Operators	Types, conversions, expressions and operators
4	Input & Decisions	Input/output, conditions and logical flow
5	Loops	for, while, range and loop controls
6	Data Structures	Strings, lists, tuples, sets and dictionaries
7	Functions & Errors	Functions, modules, exceptions and debugging
8	Files, OOP & Libraries	File handling, classes and useful libraries
9	Revision & Practice	Algorithms, exercises, glossary and study plan

How to use this with an AI study assistant: Ask for a simpler explanation, request a quiz on the current topic, paste code for debugging, or ask for another example. Try solving questions before viewing solutions.

Learning goal: Understand core Python concepts and write small programs confidently. Use this guide alongside class notes and hands-on coding.

1. Python Foundations

Python is a high-level, general-purpose programming language known for readable syntax. It supports procedural, object-oriented and functional programming styles. Python is widely used for automation, web development, data science, AI, scientific computing, testing and education.

Why Python is popular

- Readable syntax and a gentle learning curve.
- Large standard library and ecosystem of third-party packages.
- Runs on Windows, macOS and Linux.
- Useful for quick scripts as well as larger applications.

Limitations to remember

Python may be slower than compiled languages for some CPU-heavy tasks. Dependency management and runtime errors also require care. Performance can often be improved by choosing good algorithms or using optimized libraries.

Getting started

Install Python 3 from the official Python website, then use IDLE, VS Code, PyCharm or an online notebook. Save source files with the **.py** extension and run them using Python. Keep project files in a dedicated folder.

Your first program

```
print("Hello, world!")
```

print() displays information. Text goes inside quotation marks. Python is case-sensitive: name and Name are different identifiers. Indentation defines code blocks.

Syntax essentials

- Comments begin with #.
- Use consistent indentation, commonly four spaces.
- Use descriptive names such as total_marks.
- Strings can use single or double quotes.
- A colon introduces blocks after statements such as if, for and def.

```
# A comment
print("Learning Python")
```

AI study prompt: "Explain Python syntax to a beginner and quiz me with three questions, one at a time."

2. Variables, Data Types & Operators

A **variable** is a name that refers to a value. Python is dynamically typed, so you do not normally declare a variable's type before using it.

```
student = "Maya"
age = 19
height = 1.65
is_enrolled = True
print(student, age)
```

Common built-in data types

Type	Example	Purpose
int	42	Whole number
float	3.14	Decimal number
str	"Python"	Text
bool	True / False	Logical value
list	[2, 4, 6]	Ordered, mutable collection
tuple	(2, 4, 6)	Ordered, immutable collection
set	{2, 4, 6}	Unique elements
dict	{"name": "Maya"}	Key-value pairs

Naming and type conversion

Names may contain letters, digits and underscores, but cannot begin with a digit or be a Python keyword. Convert values when necessary: int("12"), float("2.5"), and str(42). Invalid conversions may raise ValueError.

Operators

Arithmetic: + - * / // % ** (add, subtract, multiply, divide, floor divide, remainder, exponent). **Comparison:** == != < > <= >=. **Logical:** and, or, not. **Assignment:** =, +=, -=, *=. **Membership:** in, not in.

```
a = 10
b = 3
print(a / b) # 3.333...
print(a // b) # 3
print(a % b) # 1
print(a ** b) # 1000
```

Remember: = assigns a value; == tests equality. Use parentheses to clarify complex expressions.

3. Input, Output & Decisions

Output: print() displays values. **Input:** input() reads text from the user and returns a string. Convert it before numeric calculations.

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
print(f"Hello, {name}. Next year you will be {age + 1}.")
```

F-strings

An f-string begins with f before the quote. Expressions inside curly braces are evaluated. For example, f"Total: {price * quantity:.2f}" formats a number to two decimal places.

Conditional statements

Use if, elif and else to choose between paths. Conditions evaluate to true or false; indentation defines each branch.

```
mark = int(input("Enter mark: "))
if mark >= 90:
grade = "A"
elif mark >= 75:
grade = "B"
elif mark >= 50:
grade = "C"
else:
grade = "Needs improvement"
print("Grade:", grade)
```

Truth values and logic

Values such as 0, None, empty strings and empty collections are falsey; most other values are truthy. Combine conditions with and, or and not.

```
age = 20
has_id = True
if age >= 18 and has_id:
print("Entry permitted")
else:
print("Entry not permitted")
```

Common mistakes

- Using = instead of == in a comparison.
- Forgetting that input() returns a string.
- Missing a colon after a conditional statement.
- Incorrect indentation or poorly chosen condition boundaries.

Mini task: Ask for a number and print whether it is positive, negative or zero. Then test values such as -1, 0 and 1.

4. Loops & Iteration

Loops repeat code. Use a for loop to iterate over a sequence or iterable; use while when repetition depends on a condition.

for loop and range()

```
for number in range(1, 6):
print(number)

# range(start, stop, step); stop is excluded
for number in range(0, 10, 2):
print(number)
```

range(1, 6) produces 1 through 5. The optional step controls the increment. You can iterate through strings, lists, tuples and dictionaries.

while loop

```
count = 3
while count > 0:
print(count)
count -= 1
print("Go!")
```

Ensure the condition eventually becomes false, or the program may loop forever. Update the controlling variable when appropriate.

Loop control

• break exits the nearest loop.
• continue skips to the next iteration.
• pass is a placeholder that does nothing.
• A loop's optional else runs if it finishes without being terminated by break.

```
for n in range(1, 8):
if n == 5:
continue
print(n)
```

Accumulators and nested loops

```
total = 0
for n in range(1, 6):
total += n
print("Sum =", total) # 15

for row in range(3):
for col in range(4):
print("*", end=" ")
print()
```

An accumulator stores a running result. Nested loops are useful for grids and tables, but may be expensive for large inputs.

Loop checklist: Identify what repeats, choose a loop, initialize state, define the stopping condition, update state, and test zero, one and many iterations.

Practice: Print a multiplication table; sum even numbers from 1 to 100; make a password prompt with a limited number of attempts.

5. Core Data Structures

Data structures organize values. Choosing the right structure makes code easier to understand and can improve efficiency.

Strings

```
text = "Python"
print(text[0]) # P
print(text[-1]) # n
print(text[1:4]) # yth
print(text.lower())
print(text.replace("Py", "J"))
```

Strings are ordered and immutable. Indexing starts at 0; negative indexes count from the end. Slicing uses [start:stop:step], excluding the stop position. Useful methods include strip(), split(), join(), find(), replace() and lower().

Lists

```
scores = [78, 91, 84]
scores.append(95)
scores[0] = 80
scores.sort()
print(scores)
print(len(scores), max(scores), sum(scores))
```

Lists are ordered and mutable. Methods include append(), extend(), insert(), remove(), pop(), sort() and reverse(). A list comprehension: [n*n for n in range(5)].

Tuples, sets and dictionaries

Tuple: ordered and immutable, e.g. point = (3, 4).
Set: unique hashable elements, e.g. unique = {1, 2, 3}; useful for uniqueness and membership checks.
Dictionary: maps keys to values, e.g. student = {"name": "Maya", "mark": 92}. Use student.get("age", 0) for a default when a key is missing.

```
numbers = [1, 2, 2, 3]
unique_numbers = set(numbers)
profile = {"name": "Maya", "mark": 92}
profile["passed"] = True
for key, value in profile.items():
print(key, value)
```

Quick choice guide

Use a list for an ordered changeable sequence; a tuple for a fixed sequence; a set for uniqueness; and a dictionary for labeled key-value data. Sets do not support positional indexing.

6. Functions, Modules & Error Handling

A function is a reusable block of code. Functions reduce repetition, clarify intent and make programs easier to test.

Defining and calling functions

```
def greet(name):
"""Return a greeting."""
return f"Hello, {name}!"

message = greet("Maya")
print(message)
```

Parameters receive values; arguments are passed during a call. return sends a result back. A function without a return value explicitly supplied returns None.

Default, keyword and flexible arguments

```
def power(base, exponent=2):
return base ** exponent

print(power(4))
print(power(base=2, exponent=5))

def add_all(*numbers):
return sum(numbers)
print(add_all(1, 2, 3, 4))
```

Avoid mutable default arguments such as items=[]. Use None and create a fresh list inside the function when needed.

Scope and modules

Variables created inside a function are normally local. A module is a Python file that can be imported. The standard library provides tools without separate installation.

```
import math
from random import randint
print(math.sqrt(81))
print(randint(1, 6))
```

Errors and exceptions

A syntax error prevents code from being parsed; a runtime exception occurs during execution; a logic error produces an incorrect result. Read tracebacks carefully to locate the exception and relevant line.

```
try:
value = int(input("Enter an integer: "))
print(100 / value)
except ValueError:
print("Please enter a valid integer.")
except ZeroDivisionError:
print("The number must not be zero.")
finally:
print("Finished.")
```

Catch specific exceptions rather than hiding all errors. Use small test cases, checkpoints and a debugger to find causes.

AI study prompt: "Review this function, explain each line, identify edge cases, and give hints before showing a corrected version."

7. Files, OOP & Useful Libraries

File handling

Use open() to read or write files. A with block closes a file automatically. Modes: r read, w write/overwrite, a append, and x create only if absent.

```
with open("notes.txt", "w", encoding="utf-8") as file:
file.write("Practice Python every day.\n")

with open("notes.txt", "r", encoding="utf-8") as file:
content = file.read()
print(content)
```

The standard library includes json and csv for structured data. In real applications, handle missing files and permission errors.

Object-oriented programming

A **class** defines attributes and behavior; an **object** is an instance. __init__ initializes an instance, and self refers to that instance. Inheritance lets one class extend another.

```
class Student:
def __init__(self, name, mark):
self.name = name
self.mark = mark

def passed(self):
return self.mark >= 50

student = Student("Maya", 82)
print(student.name, student.passed())
```

Core OOP ideas: encapsulation (organizing data and behavior), abstraction (exposing essential operations), inheritance (extending behavior), and polymorphism (a common interface with different implementations).

Useful libraries

math mathematical functions; **datetime** dates and times; **random** pseudo-random choices; **pathlib** filesystem paths; **json** JSON data; **statistics** basic statistics. Third-party tools: **NumPy** arrays, **pandas** data analysis, **Matplotlib** charts, **scikit-learn** machine learning, and **Flask/FastAPI** web APIs.

Virtual environments

```
python -m venv .venv
# Activate the environment using your OS instructions
python -m pip install package-name
```

A virtual environment isolates project dependencies. Install packages you trust and never put passwords or API keys directly into source code.

8. Revision, Algorithms & Practice

Problem-solving workflow

1. Understand the problem and inputs/outputs. 2. Break it into smaller steps. 3. Write pseudocode. 4. Implement a simple correct solution. 5. Test normal, boundary and invalid inputs. 6. Refactor repeated logic. 7. Comment when explaining why a choice was made.

Algorithm example: find the largest number

```
numbers = [12, 4, 29, 17]
largest = numbers[0]
for number in numbers:
if number > largest:
largest = number
print(largest) # 29
```

The example scans the list once, so work grows in proportion to input size: O(n). Big-O notation describes how resource use grows as input grows. O(1) is constant, O(log n) grows slowly, O(n) is linear, and $O(n^2)$ often appears in nested loops over the same input.

Practice questions

1. Calculate a rectangle's area from user input.
2. Check whether a number is even or odd.
3. Print numbers divisible by 3 from 1 to 100.
4. Count vowels in a string.
5. Find the maximum and minimum in a list.
6. Remove duplicates while preserving order.
7. Write a factorial function for non-negative integers.
8. Read a text file and count its lines.
9. Store student marks in a dictionary and show the highest scorer.
10. Create a bank-account class with deposit and withdraw methods.

Hints for self-check

1. area = length * width. 2. Test n % 2 == 0. 3. Use a loop and condition. 4. Convert text to lowercase and test for vowels. 5. Try min() and max(). 6. Track seen values in a set. 7. Use a loop with a correct starting value. 8. Iterate through lines. 9. Compare dictionary values. 10. Validate that a withdrawal does not exceed the balance.

Glossary

Algorithm: finite steps to solve a problem. **Bug:** a code defect. **Iteration:** repeating steps. **Iterable:** an object usable in a loop. **Parameter:** named input in a function definition. **Exception:** runtime error condition. **Library:** reusable code. **API:** defined communication between software components.

Suggested 7-day plan

Day 1: syntax and variables. Day 2: operators and conditions. Day 3: loops. Day 4: strings and collections. Day 5: functions and exceptions. Day 6: files and OOP. Day 7: practice problems and explain solutions aloud.

Use your AI study assistant wisely: Ask for hints first, request quizzes on weak topics, and verify explanations with official documentation. Always run code and check outputs.

Reference: Python documentation — https://docs.python.org/3/